

SecureShield: Role-Based Access Control (RBAC) API Report

1. Project Overview and Implementation

SecureShield is a backend security simulation developed to implement a robust authentication and authorization framework. The system is built using the Flask framework, incorporating industry-standard libraries such as Flask-Bcrypt for cryptographic operations and PyJWT for identity management. The primary objective was to establish a secure environment that strictly adheres to the Principle of Least Privilege by segregating user capabilities based on defined roles.

The identity flow begins with a secure registration system where passwords undergo salting and hashing using the Bcrypt algorithm before being persisted to a JSON-based local database. Upon a successful login attempt, the server issues a signed JSON Web Token (JWT) containing the user's identity and role. To protect sensitive endpoints, a middleware decorator was implemented to intercept requests, verifying the token's validity, expiration, and signature. This ensures that only authenticated users can access the system, while a specialized "blacklist" mechanism handles token revocation during logout to prevent replay attacks.

2. Security Discussion

The Necessity of Salting against Rainbow Table Attacks: Salting is a critical defensive measure used to mitigate the effectiveness of pre-computed dictionary attacks known as Rainbow Tables. A Rainbow Table contains millions of pre-calculated hashes for common passwords, allowing an attacker to quickly reverse a captured hash without

significant computational effort. By appending a unique, random "salt" to each password before the hashing process, the system ensures that even if two users share the same password, their final stored hashes will be entirely different. This effectively invalidates pre-computed tables, forcing attackers to perform computationally expensive brute-force attacks for every individual user record.

Risks of Storing Sensitive Data in JWT Payloads: A common security misconception is that JSON Web Tokens provide data confidentiality. In practice, the payload of a JWT is only Base64Url encoded, meaning it can be easily decoded and read by any party that intercepts the token without requiring a secret key. Therefore, storing sensitive information such as passwords, social security numbers, or internal system credentials within the payload poses a significant data exposure risk. While the token's signature ensures data integrity—preventing unauthorized tampering—it provides no protection against eavesdropping. Consequently, payloads should only contain non-sensitive identifiers like user IDs or roles.

3. Submission Links

GitHub Repository: www.github.com/...

YouTube Demo Video: [www.youtube.com/...](http://www.youtube.com/watch?v=UW3333333333)

Utku Ala - 220208015

Ömer Seyfettin Yıldız - 220208046

Yavuz Gürkan Kuşca - 230208049